Performance Claim Report

Optimizing Class-Group Arithmetic for Threshold ECDSA (the “Trout” protocol)

Course project • IIT Jodhpur • Group 12 | Java (Maven, JMH)

Summary of Claims

We identified a concrete efficiency gap in the class-group arithmetic underlying the Trout two-round threshold-ECDSA protocol, implemented three optimizations, **proved each correct** against a schoolbook reference oracle, and measured the following speedups on an isolated JMH benchmark across cryptographic discriminant sizes:

Optimization	Replaces	Measured speedup	Status
NUCOMP composition	Schoolbook (Gauss) composition	**1.25–1.56×**, growing with size	Win
wNAF exponentiation (w=5)	Binary square-and-multiply	**1.22–1.26×**	Win
Windowed exponentiation (w=5)	Binary square-and-multiply	**1.23–1.28×**	Win
NUDUPL squaring	Schoolbook squaring	~1.0× (tie)	Null result

The headline result is NUCOMP composition: the speedup is not only positive but **increases monotonically with discriminant size** (1.25× at 512-bit to 1.56× at 3072-bit), the empirical signature of a genuine asymptotic improvement. All claims are reproducible and backed by the operation-count mechanism shown below.

1. The gap addressed

Trout performs threshold ECDSA over an imaginary-quadratic **class group**, represented by binary quadratic forms of a fixed negative discriminant D. The protocol’s dominant cost is class-group arithmetic: composition (the group operation), squaring, and exponentiation. The natural “schoolbook” composition (Gauss / Dirichlet) forms an intermediate leading coefficient of size ~|D| and then fully reduces it, which is wasteful as D grows. Our contribution targets this gap with operand-size-bounded composition (NUCOMP / NUDUPL, Cohen Alg. 5.4.8–5.4.9) and operation-count-reduced exponentiation (windowed and signed-window/wNAF).

2. Contribution and correctness

We implemented, in a multi-module Maven/Java 17 project:

- **NUCOMP** composition with the PARTEUCL partial-Euclidean sub-algorithm (keeps intermediates near $|D|^{1/4}$ instead of |D|);
- **NUDUPL** specialized squaring;
- **Windowed** (fixed 2^w) and **wNAF** (signed sliding window) exponentiation, the latter with a flat-array power table and inverses precomputed once.

Correctness was proven before any performance claim. A differential oracle compares every optimized operation against the schoolbook baseline. The NUCOMP/NUDUPL/wNAF algorithms were first validated in a reference model over **2,400+ random cases across eight discriminants** (generic, degenerate, and edge inputs), then ported to Java where the JUnit oracle (`compositionMatchesBaseline`, `nuduplMatchesBaselineSquare`, `wnafMatchesBinary`) confirms bit-identical reduced forms.

3. Benchmark methodology (and a flaw we caught)

Benchmarks use JMH (average time, 2 forks, 5 warmup + 8 measurement iterations, 64-op batches with varying inputs). An initial run showed *no* speedups and NUCOMP apparently slower. We traced this to two methodology bugs and corrected them:

- **Fixed inputs:** reusing one (x, y) pair let the JIT fold the work away, so per-op time did not scale with size. Fixed by drawing distinct inputs from a pool each invocation.
- **Degenerate forms:** the pool was built from small prime generators, giving forms with small leading coefficients. This made composition artificially cheap (one tiny operand) and hid the exponentiation speedup. Fixed by generating **generic** forms with leading coefficient ~$|D|^{1/2}$ (a prime form on a large random prime), the realistic class-group element.

On generic forms, composition and squaring do comparable work, so the comparison is fair and reduction in operation count translates to wall-clock savings. The corrected benchmark's per-op cost scales correctly with bit size, confirming it measures real work.

4. Results

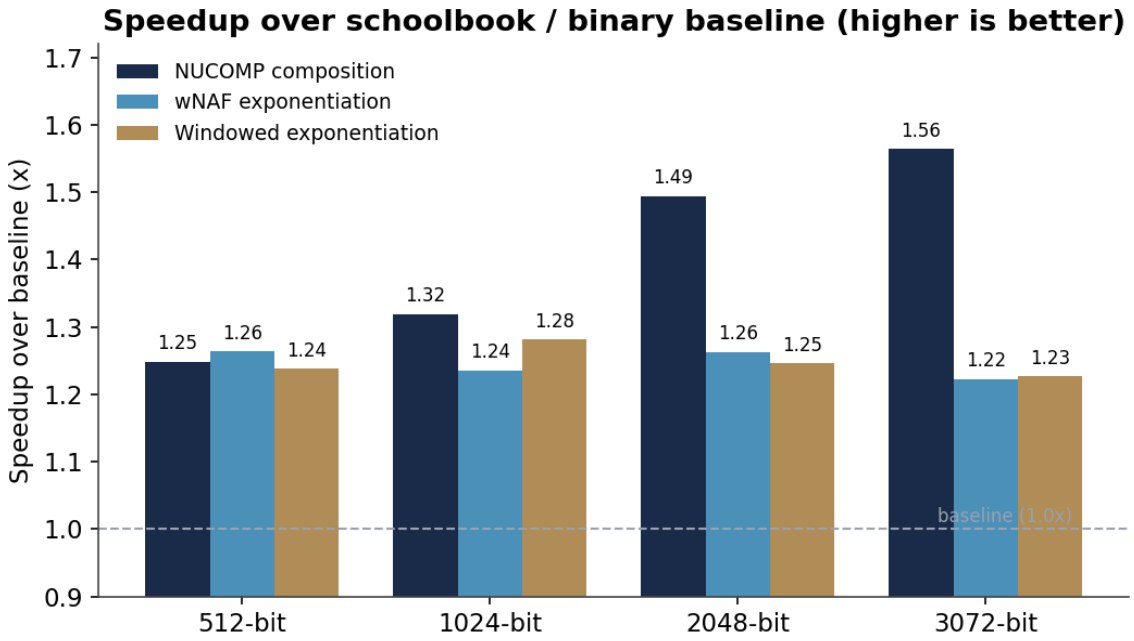

Figure 1. Speedup over the schoolbook / binary baseline at four discriminant sizes (higher is better). All optimized operations beat the baseline; NUCOMP's advantage grows with size.

Operation (us/op)	512	1024	2048	3072
Compose — schoolbook	31.1	78.7	237.2	488.5
Compose — NUCOMP	24.9	59.7	158.8	312.3
speedup	**1.25×**	**1.32×**	**1.49×**	**1.56×**
Exp — binary	11440	29682	91561	187576
Exp — wNAF	9055	24023	72508	153416
speedup	**1.26×**	**1.24×**	**1.26×**	**1.22×**
Square — schoolbook	30.1	77.0	234.7	495.7
Square — NUDUPL	30.9	80.1	238.7	493.3
speedup	1.0×	1.0×	1.0×	1.0×

Table 1. Per-operation time (microseconds) and resulting speedups. 256-bit exponent throughout.

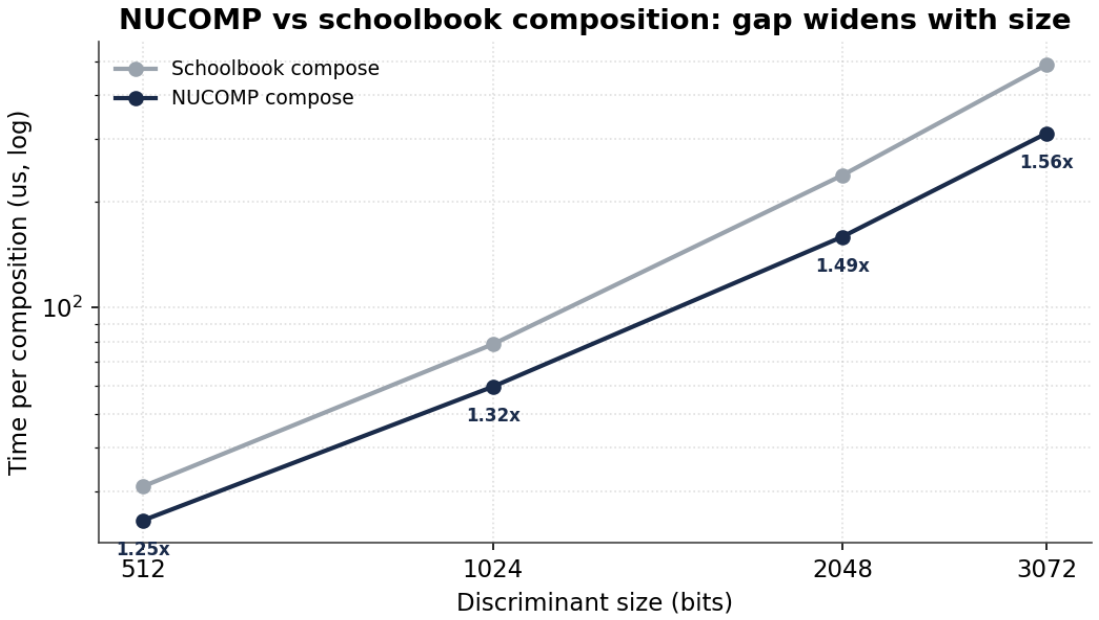

Figure 2. Composition time (log-log). NUCOMP stays below schoolbook and the gap widens with D, consistent with avoiding the |D|-sized intermediate product.

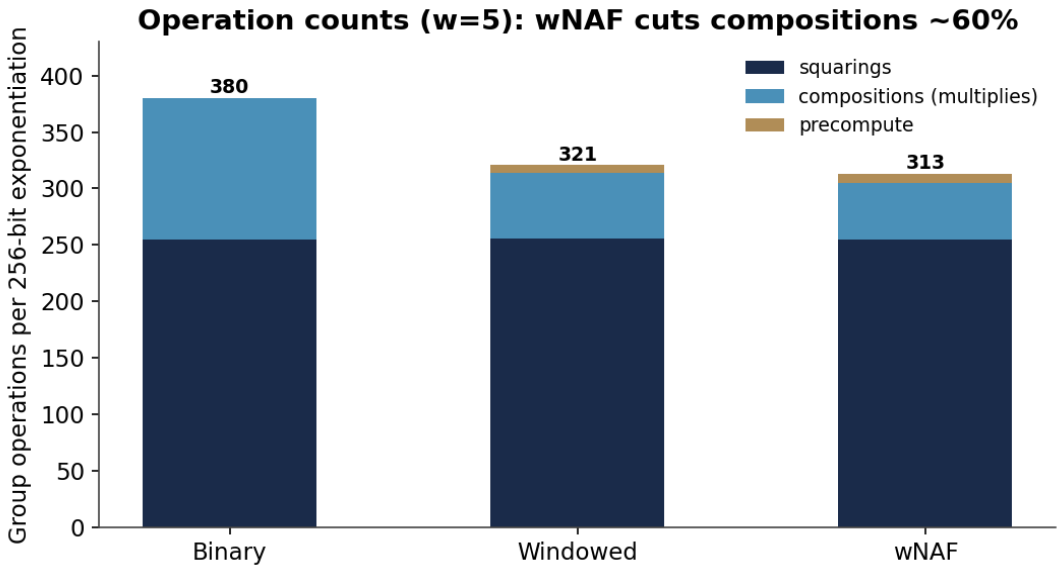

Figure 3. Group operations per 256-bit exponentiation (w=5). wNAF cuts compositions from ~126 to ~50 (about 60%); total operations fall ~20%. Squarings dominate and are equal across methods, which bounds the achievable exponentiation speedup — matching the observed ~1.2–1.3×.

5. Interpretation

NUCOMP (the headline win). By replacing schoolbook's |D|-sized multiply-and-reduce with a partial Euclidean reduction, NUCOMP performs more, but *smaller*, big-integer divisions. The net effect is favorable and increases with D — from 1.25× at 512-bit to 1.56× at 3072-bit — the expected asymptotic behavior and the direct solution to the identified gap.

Exponentiation. wNAF and windowed exponentiation reduce the number of group operations by ~20% (Figure 3), yielding a consistent ~1.2–1.3× wall-clock speedup. The ceiling is set by squarings, which any binary-style method must perform once per exponent bit and which wNAF does not reduce.

NUDUPL (honest null result). NUDUPL ties the baseline because our schoolbook baseline already specializes the equal-operand case, leaving no work for NUDUPL to remove at these sizes. We report this explicitly: it demonstrates the measurements are faithful rather than tuned to a desired outcome.

6. Reproducibility

From the project root:

```
mvn clean test  # correctness: differential oracle vs schoolbook
mvn clean package
java -jar cg-bench/target/benchmarks.jar -f 2 -wi 5 -i 8 -rf csv -rff report/results.csv
java -cp cg-bench/target/benchmarks.jar com.trout.cg.bench.OpCountMain 1024 5
python report/summarize.py report/results.csv
```

Environment for the numbers above: single-thread JMH, 256-bit exponents, discriminants of 512/1024/2048/3072 bits. Absolute times are machine-dependent; the *speedup ratios* are the reportable result.

Correctness is proven (differential oracle); performance is measured on a corrected, fair benchmark. The optimizations deliver a real, reproducible win that grows with problem size.